

MONITOR Z · 00

Ruta y glosario

Cuaderno de estudio · versión del 2026-10-05

Margen derecho y renglones libres para tus notas. Las láminas son las mismas de la carpeta **_visual**. Si una lección se corrige, se regenera este cuaderno con la fecha nueva.

Ruta de aprendizaje

Doce módulos, de "qué es un demonio" a "un STC que avisa al instante y el portal que aprende de él". **Cada módulo termina con algo que corre en DESA**, no sólo con teoría. El orden sigue el plan ya acordado en IDEA-005:

1. Primero **validar sin puerto ni APF** (modo spool).
2. Después agregar el socket.
3. Recién al final convertirlo en STC autorizado con eventos ENF 70.

Así cada paso depende sólo de lo que ya se probó.

Mód.	Tema	Qué aprendés	Qué queda implementado / medido	Requiere de sistemas
0	<i>L00 - Que es un demonio en z-OS y que vamos a construir</i>	Demonio en Unix vs. STC y job largo en z/OS, las piezas del monitor y los 3 niveles (polling, job que duerme, eventos)	El mapa y el diagnóstico de dónde estamos	—
1	<i>L01 - Anatomía de un programa Assembler - el prologo de ZTKMONS</i>	Convención de enlace, save areas, registros base, PARM, AMODE/RMODE, DCB	Leer y explicar el prólogo y el epílogo de ZTKMONS	—
2	<i>El bucle: dormir sin gastar CPU</i>	STIMER, WAIT / POST, el ECB como semáforo (el sem_timedwait de SOPES2 en z/OS)	Un programa mínimo que despierta cada N s, escribe la hora y termina a los M min. Medir su CPU	—
3	<i>Hablar con JES: el SSI</i>	IEFSSREQ, SSOB/SSIB, función 80 (extended status), STATTERS y STATVRBO, las áreas que devuelve y cómo liberarlas	Leer ZTKJACT y la rutina LLAMASSI de ZTKMONS	—

4	ZTKMONS v0 en DESA (modo spool)	Ensamblar y enlazar (ASMA90, binder), leer los errores, correr la demo de vivo\host\LEEME.md	Primera validación real: spool_a_portal.py <JOBID> con ✓/X, y las 5 preguntas abiertas contestadas	—
5	<i>Diseño del protocolo</i>	Por qué texto EBCDIC con X'15', idempotencia, reconexión (ACT, HIST), latido, versionado	Revisar PROTOCOLO.md con lo que enseñó la demo; v1.1 si hace falta	—
6	<i>Sockets en z/OS</i>	API EZASMI/EZASOKET: SOCKET, BIND, LISTEN, ACCEPT, READ y WRITE; bloqueo y plazos (el CONNECT de 77 s de ZTKAVISA)	ZTKMON escuchando en un puerto de DESA y el puente.py real conectado (sin simulador)	Puerto TCP
7	<i>Ser un STC de verdad</i>	PROCLIB, START, comandos del operador (F y P con EXTRACT COMM, CIB y QEDIT), ESTAE/recuperación, reentrancia	ZTKMON arranca con S ZTKMON, atiende F ZTKMON, ESTADO y para limpio con P ZTKMON	STC + PROCLIB
8	<i>Eventos de verdad: ENF 70</i>	APF y estado supervisor, ENFREQ LISTEN, listen exit y POST del ECB, el mapeo IAZENF70, cuidados de un exit (corto, sin I/O)	Aviso instantáneo de INI y FIN sin polling. Comparar latencia y CPU con el modo polling	Librería APF
9	<i>¿Quién tiene el archivo? (ENQ y GQSCAN)</i>	Recursos SYSDSN, ENQ y DEQ, GQSCAN para ver dueños y esperas, sin encolarse nunca	Alerta de contención (IDEA-008): el caso FDCICS02/CICSFILE del 05-10 visto en vivo	— (probar si alcanza estado problema)
10	<i>El lado PC: puente, portal y ML en línea</i>	puente.py, vivo.py, umbral de colgado con Kaplan-Meier, por qué no se aprende de la propia anomalía	Regresión prueba_e2e.py 11/11 contra el STC real	—

11	<i>Operación y pase</i>	Medir CPU por vuelta y por evento, gestión de cambios, runbook, qué pasa si el monitor se cae	Informe de la demo y propuesta para PROD (sigue siendo de sólo lectura)	Gestión de cambios
----	-------------------------	---	---	--------------------

Ruta de aprendizaje: 12 módulos

Monitor z · plan

Mód.	Tema	Qué queda funcionando	Requiere	Estado
0	Qué es un demonio en z/OS	El mapa	—	escrita
1	Anatomía Assembler: prólogo y epílogo	Práctica P01: ZTKHOLA	—	escrita
2	Dormir sin gastar CPU: STIMER, WAIT/POST, ECB	Un bucle con su CPU medida	—	
3	Hablarle a JES: SSI 80	Leer ZTKJACT y LLAMASSI	—	
4	ZTKMONS v0 en DESA (modo spool)	Primera validación real + las 5 preguntas	—	
5	Diseño del protocolo	PROTOCOLO.md revisado	—	
6	Sockets en z/OS (EZASMI)	Monitor conectado al portal real	puerto TCP	
7	Ser un STC: F/P, ESTAE, reentrancia	S / F / P ZTKMON	STC + PROCLIB	
8	Eventos de verdad: ENF 70	Aviso instantáneo; comparar con el polling	APF	
9	¿Quién tiene el archivo? ENQ/GQSCAN	Contención en vivo (IDEA-008, FDCICS02)	—	
10	Lado PC: puente, portal, ML en línea	Regresión 11/11 contra el monitor real	—	
11	Operación y pase	Informe y propuesta (sigue siendo de sólo lectura)	gestión de cambios	

Analogía: subir una montaña por campamentos

Cada módulo es un campamento: no se sube al siguiente sin haber armado la carpa en el anterior. Los campamentos 0–5 se hacen **sin pedir permisos**; del 6 en adelante hace falta que **sistemas** abra la ruta (puerto, STC, APF).

Fuente: Ruta de aprendizaje (cerebro monitor-z)

Lámina R-01_ruta_de_aprendizaje

Estado

- [x] Teoría de los 12 módulos escrita (2026-10-05).
- [] Prácticas: P01 (ZTKHOLA) es la próxima.

Dónde está el código hoy: ZTKMONS v0 (944 líneas) está **escrito y nunca se ensambló**. El puente, el simulador y el portal andan: la regresión dio 11/11 el 2026-09-30. Detalle en [L00 - Que es un demonio en z-OS y que vamos a construir](#).

Criterio propio: el orden $2 \rightarrow 3 \rightarrow 4$ es para que el primer ensamblado (mód. 4) ya lo entiendas línea por línea. Si preferís ensamblar primero y entender después, se puede adelantar el 4.

Glosario

Una línea por término. La lección donde se explica a fondo va entre paréntesis.

Término	Qué es
Address space	El "proceso" de z/OS: un espacio de memoria propio con sus tareas. Cada job y cada STC tiene el suyo (L00, L07)
AMODE / RMODE	Con cuántos bits direcciona el programa (24/31/64) y dónde puede cargarse (bajo o sobre la línea de 16 MB) (L01)
APF	<i>Authorized Program Facility</i> : librerías cuyos programas pueden pedir servicios autorizados (estado supervisor). Lo necesita ENF 70 (L08)
Base + desplazamiento	Forma de direccionar: registro base + 12 bits (0–4095), por eso un registro base cubre 4 KB (L01)
CIB	<i>Command Input Buffer</i> : por donde le llegan a un STC los comandos F (MODIFY) y P (STOP) del operador (L07)
DCB	<i>Data Control Block</i> : describe un archivo para QSAM/BSAM. El de QSAM tiene que estar bajo la línea (L01)
DSECT	Plantilla de un área de memoria, sin reservarla: se "apoya" sobre una dirección con USING (L01, L03)
EBCDIC / X'15'	Código de caracteres de z/OS. X'15' (NL) es el fin de línea del protocolo ZTKMON (L05)
ECB	<i>Event Control Block</i> : una palabra que hace de semáforo. WAIT duerme hasta que alguien hace POST (L02)
ENF 70	Señal de JES2 cuando un job cambia de estado (SELECT, DESELECT, CHANGE, PURGE). Se escucha con ENFREQ LISTEN, que requiere autorización (L08)
ENQ / GQSCAN	ENQ serializa recursos (por ejemplo, SYSDSN + nombre de dataset). GQSCAN lista quién los tiene y quién espera (L09)
ESTAE	Rutina de recuperación: se ejecuta si el programa abenda, para limpiar y avisar (L07)
Iniciador	Address space de JES que ejecuta jobs batch de ciertas clases. Un job largo lo ocupa todo el tiempo (L00)
JES2	El subsistema que recibe, encola, ejecuta y guarda la salida (spool) de los jobs (L03)

Reentrante	Programa que no modifica su propio código ni sus datos en la CSECT. Varias tareas pueden usar la misma copia (L01, L07)
Save area	Bloque de 72 bytes donde se guardan los registros del que llama; se encadenan con HSA y LSA (L01)
SSI / IEFSSREQ	<i>Subsystem Interface</i> : la forma de pedirle cosas a JES desde un programa (L03)
SSI 80	Función 80 del SSI, <i>extended status</i> : estado de jobs. Variantes STATTERS (resumen) y STATVRBO (detalle). No requiere APF (L03)
SSOB / SSIB	Bloques de control del pedido SSI: qué función se pide (SSOB) y a qué subsistema (SSIB) (L03)
STC	<i>Started task</i> : programa que se arranca con <code>S nombre</code> , vive en su address space y no usa un iniciador batch. Es el "demonio" de z/OS (L00, L07)
STIMER	Macro para esperar un intervalo (<code>STIMER WAIT</code>) o programar una salida por tiempo (L02)
USING	Promesa al ensamblador sobre qué dirección va a tener un registro base (L01)
WTO	<i>Write To Operator</i> : mensaje a la consola y al SYSLOG (L01, L07)

Término	Qué es	Lección
Address space	El "proceso" de z/OS: memoria propia con sus tareas. Cada job y cada STC tiene el suyo	L00, L07
AMODE / RMODE	Con cuántos bits direcciona (24/31/64) y dónde puede cargarse (bajo o sobre los 16 MB)	L01
APF	Librerías autorizadas: sus programas pueden pedir servicios de estado supervisor. Lo necesita ENF 70	L08
Base + desplaz.	Registro base + 12 bits (0–4095): un registro base cubre 4 KB	L01
CIB	Por donde le llegan a un STC los comandos F (MODIFY) y P (STOP)	L07
DCB	Describe un archivo para QSAM/BSAM; el de QSAM va bajo la línea	L01
DSECT	Plantilla de un área, sin reservarla; se apoya con USING	L01, L03
EBCDIC · X'15'	Código de caracteres de z/OS · fin de línea (NL) del protocolo	L05
ECB	Palabra que hace de semáforo: WAIT duerme hasta que otro hace POST	L02
ENF 70	Aviso de JES2 cuando un job cambia de estado; se escucha con ENFREQ LISTEN (autorizado)	L08
ENQ · GQSCAN	Serializar un recurso (SYSDSN + dataset) · listar quién lo tiene y quién espera	L09
ESTAE	Rutina de recuperación si el programa abanda	L07
Iniciador	Address space de JES que ejecuta jobs batch de ciertas clases	L00
JES2	Recibe, encola, ejecuta y guarda la salida (spool) de los jobs	L03
Reentrante	No modifica su código ni sus datos en la CSECT; varias tareas comparten la copia	L01, L07
Save area	72 bytes con los registros del que llama; se encadenan con HSA y LSA	L01
SSI · IEFSSREQ	Subsystem Interface: la forma de pedirle cosas a JES	L03
SSI 80	Extended status de jobs (STATTERS resumen, STATVRBO detalle); sin APF	L03
SSOB · SSIB	Bloques del pedido SSI: qué función (SSOB) y a qué subsistema (SSIB)	L03
STC	Started task: se arranca con S, address space propio, sin iniciador batch	L00, L07
STIMER	Esperar un intervalo (STIMER WAIT) o programar una salida por tiempo	L02
USING	Promesa al ensamblador del valor de un registro base	L01
WTO	Write To Operator: mensaje a la consola y al SYSLOG	L01, L07

Fuente: Glosario (cerebro monitor-z) · manuales de IBM citados en cada lección

Lámina G-01_glosario

Relacionado: [00 - Monitor z \(MOC\)](#) · [L00 - Que es un demonio en z-OS y que vamos a construir](#) · [L01 - Anatomía de un programa Assembler - el prologo de ZTKMONS](#).

 Fuente: definiciones resumidas de los manuales de IBM citados en cada lección (Assembler Services Guide, Using the Subsystem Interface, Authorized Assembler Services Reference, HLASM Language Reference).

